

Exercício de Revisão de Associação Todo-Parte

Daniel Facciolo Pires • 20 de mar.

Data de entrega: 27 de mar., 23:59

Parte 1: Classe de Associação (Relacionamentos com Atributos)

Exercício 1: Sistema de Clínica Médica

Crie um sistema para gerenciar consultas.

- Classes: Medico (id, nome, especialidade) e Paciente (id, nome, cpf).
- Classe de Associação: Consulta. Ela deve conter a Data (use LocalDateTime), o ValorDaConsulta e as referências para o Médico e o Paciente.
- **Desafio:** No main, crie um médico e um paciente, e registre uma consulta para o dia 20/05/2026.

Exercício 2: Sistema de Biblioteca

Modele o empréstimo de livros.

- Classes: Leitor (id, nome) e Livro (id, titulo, autor).
- Classe de Associação: Empréstimo. Deve conter a dataEmprestimo, a dataDevolucaoPrevista e o vínculo entre o Leitor e o Livro.
- **Regra:** O método toString() do empréstimo deve mostrar os dados do leitor e o título do livro emprestado.

Parte 2: Apenas Agregação (Independência entre as partes)

Exercício 3: Time de Basquete (Inspirado no Franca Basquete)

- Classes: Atleta (id, nome, posicao) e Time (id, nome, tecnico).
- O Time possui um ArrayList<Atleta>.
- **Regra:** Implemente o método void contratarAtleta(Atleta a). No main, prove que se o objeto Time for anulado, o objeto Atleta continua existindo.

Exercício 4: Projeto de Software

- Classes: Programador (id, nome, linguagemPrincipal) e Projeto (id, nomeProjeto).
- O Projeto agrega vários programadores.
- **Regra:** Crie um método para listar todos os programadores que estão trabalhando naquele projeto específico.

Parte 3: Apenas Composição (Dependência Existencial)

Exercício 5: Computador e Seus Componentes

- Classes: Processador (marca, modelo, frequencia) e Computador (id, marca).
- Um Computador é composto por **um** Processador.
- **Regra:** O processador deve ser instanciado **dentro** do construtor da classe Computador. Se o computador for deletado, o processador também deve ser.

Exercício 6: Edifício e Apartamentos

- Classes: Apartamento (numero, andar) e Edificio (nome, endereco).
- O Edificio possui um ArrayList<Apartamento>.
- **Regra:** Implemente o método void construirApartamento(int num, int andar). Os dados do apartamento são passados como tipos primitivos, e o objeto é criado dentro do Edificio.

Parte 4: Agregação e Composição Combinadas

Exercício 7: Sistema de Vendas de Supermercado

Este exercício combina tudo o que vimos até agora: **Agregação**: Crie a classe Produto (id, nome, preco). O Produto existe no estoque da loja independente de qualquer venda. **Agregação**: Crie a classe Cliente (id, nome). **Composição**: Crie a classe ItemVenda (id, quantidade). O item de venda só existe dentro de uma Venda. **A Classe "Todo"**: Crie a classe Venda.

- Ela agrega **um** Cliente.
- Ela é composta por um ArrayList<ItemVenda>.
- Cada ItemVenda dentro da lista, por sua vez, agrega **um** Produto.

Exercício 8: Sistema de Cine-Teatro

Objetivo: Criar um sistema que gerencie uma Sessao, relacionando-a com um Filme (Agregação) e com os Ingressos vendidos (Composição). 1. Classe Filme (Objeto Independente)

Um filme existe no catálogo do cinema, independentemente de estar em cartaz ou não.

- **Atributos**: id, titulo, genero, duracao (int - em minutos).
- **Requisito**: Construtor completo, Getters/Setters e toString().

2. Classe Ingresso (Objeto Parte)

Um ingresso só faz sentido dentro de uma sessão específica. Se a sessão for cancelada, o ingresso perde o valor e deixa de existir no sistema.

- **Atributos**: id, assento (String), tipo (String: "Inteira" ou "Meia"), preco (float).
- **Requisito**: Construtor completo e toString().

3. Classe Sessao (O "Todo" Principal)

A Sessao é o coração do sistema, onde as relações acontecem.

- **Atributos**: id, horario (use LocalDateTime), sala (int).
- **Relacionamentos**:
- **Agregação**: Um atributo do tipo Filme. (Use o método vincularFilme(Filme f) que recebe o objeto pronto).
- **Composição**: Um ArrayList<Ingresso>. (Use o método venderIngresso(int id, String assento, String tipo, float preco) que cria o objeto Ingresso internamente).

4. Classe de Teste TestaCinema

No método main, realize o fluxo: Instancie 2 objetos Filme (ex: "Batman", "Duna"). Instancie uma Sessao para a Sala 01, informando uma data e hora futura (ex: 20/07/2026 às 20h00). **Agregação**: Vincule o filme "Batman" a esta sessão. **Composição**: Simule a venda de 3 ingressos (A1, A2 e A3) usando o método de composição da classe Sessao. Imprima os dados da Sessão.